

Genuine Phantoms

$|\text{Sha}| = 4$ Through the BSD Compiler

Dr. Clifford Keeble

Bootstrap Universe Programme

Woodbridge, UK

March 2026

Paper 144 · v1.0

Abstract

We run the BSD compiler of Paper 142 on the elliptic curve 27606c1, which has analytic $|\text{Sha}| = 4$. Every ingredient — Fourier coefficients by point counting, L-value by convergent series, Tamagawa numbers by the Tate algorithm, real period by high-precision quadrature — is computed from the Weierstrass coefficients alone. The compiler outputs $|\text{Sha}| = 4.0000000000$: a perfect square. This extends the three-way comparison of Paper 143: genuine curves with $|\text{Sha}| = 1$ produce 1 (a perfect square), genuine curves with $|\text{Sha}| = 4$ produce 4 (a perfect square), and fake controlling functions produce non-integers. The compiler distinguishes genuine phantoms — Sha elements that carry topological content — from fake perturbations that carry none. The period computation for this curve requires particular care: its short Weierstrass form is nearly degenerate, with complex conjugate roots separated by only 1.7×10^{-4} , creating a spike of height 23,000 in the period integrand that standard quadrature misses entirely. The resolution — multi-point splitting through the spike zone with 80-digit intermediate arithmetic — achieves 30-digit agreement with the LMFDB value. All code is provided in Appendix A.

§1 Introduction

Paper 142 [1] established the BSD compiler: a uniform pipeline that takes Weierstrass coefficients as input and produces the BSD quotient as output. Paper 143 [2] tested the compiler with fake controlling functions and found that perturbations produce non-integer output. Together, these papers established that the compiler's integer-output mechanism requires topological input.

But Paper 143 left a gap. Its Table 1 contained two types of input: genuine (389a1, with $|\text{Sha}| = 1$) and fake (random noise and systematic bias, both non-integer). The missing case was a genuine curve with $|\text{Sha}| > 1$ — a curve whose Tate–Shafarevich group contains actual phantom elements. If the compiler produces a perfect square for such a curve, it demonstrates that the integer-output mechanism is not merely a property of trivial Sha, but a property of the compiler applied to genuine arithmetic input. Genuine phantoms produce integers. Fake phantoms do not.

The present paper fills this gap. We compute $|\text{Sha}|$ for the elliptic curve 27606c1, which has analytic $|\text{Sha}| = 4$, deriving every quantity from the Weierstrass coefficients $[1, 0, 0, -10289707, 12703497719]$ with no external input.

§2 The Curve

§2.1 Selection

We require a rank-0 curve with $|\text{Sha}| > 1$ whose bad reduction is entirely multiplicative (so that the semistable Tate algorithm of Paper 142 applies without the ~ 15 additional sub-cases needed for additive reduction at $p = 2$ and $p = 3$). Cremona’s tables [3] and the LMFDB [4] list 27606c1 as a semistable curve with rank 0, trivial torsion, and analytic $|\text{Sha}| = 4$.

§2.2 Invariants

The minimal Weierstrass equation is

$$y^2 + xy = x^3 - 10289707x + 12703497719$$

with conductor $N = 27606 = 2 \cdot 3 \cdot 43 \cdot 107$. All four primes of bad reduction have multiplicative reduction:

p	Kodaira type	Reduction	a_p	c_p
2	I_1	split multiplicative	+1	1
3	I_3	split multiplicative	+1	3

43	I_3	nonsplit multiplicative	-1	1
107	I_1	split multiplicative	+1	1

The Tamagawa product is $\prod c_p = 1 \cdot 3 \cdot 1 \cdot 1 = 3$. Torsion is trivial. The regulator is 1 (rank 0). The BSD formula therefore reduces to

$$|Sha| = L(E, 1) / (\Omega \cdot \prod c_p) = L(E, 1) / (3\Omega)$$

§3 The Computation

§3.1 Fourier coefficients

The coefficients a_p are computed by direct point counting modulo p for all primes $p \leq 5000$. The full sequence a_n for $n \leq 5000$ is then built using Hecke multiplicativity: for good primes, $a_{p^k} = a_p \cdot a_{p^{k-1}} - p \cdot a_{p^{k-2}}$; for bad primes, $a_{p^k} = (a_p)^k$. Verification against the LMFDB q -expansion confirms all eight tested coefficients:

p	a_p (computed)	a_p (LMFDB)	Status
2	+1	+1	✓
3	+1	+1	✓
5	-3	-3	✓
7	+5	+5	✓
11	+4	+4	✓
13	+4	+4	✓
17	+7	+7	✓
19	+1	+1	✓

§3.2 L-function at $s = 1$

The L-value is computed via the rapidly convergent series

$$L(E, 1) = 2 \cdot \sum_{n=1}^{\infty} (a_n/n) \cdot \exp(-2\pi n/\sqrt{N})$$

with 5000 terms, giving $L(E, 1) = 6.457030691756106$ (16-digit agreement with the LMFDB value 6.457030691756105 , error 8.88×10^{-16}). The exponential decay factor $\exp(-2\pi n/\sqrt{N}) \approx \exp(-0.0378n)$ ensures rapid convergence.

§3.3 Tamagawa numbers

The minimal discriminant is $\Delta_{\min} = -459391446 = -2 \cdot 3^3 \cdot 43^3 \cdot 107$. For each bad prime p , the Kodaira type is I_n where $n = \text{ord}_p(\Delta)$. Split/nonsplit is determined by the sign of a_p : split if $a_p = +1$, nonsplit if $a_p = -1$. For split multiplicative reduction, $c_p = n$; for nonsplit with n odd, $c_p = 1$. This gives $\prod c_p = 3$, matching the LMFDB value exactly.

§3.4 The near-degenerate period

This is where 27606c1 presents a non-trivial computational challenge. The standard approach converts to short Weierstrass form $y^2 = x^3 + Ax + B$ via the substitution $X = x + b_2/12$, $Y = (2y + a_1x + a_3)/2$. Under this substitution, the Néron differential $dx/(2y + a_1x + a_3)$ transforms to $dX/(2Y)$, so the two models have identical periods. For 27606c1:

$$A = -c_4/48 = -10289707.0208\dots, \quad B = -c_6/864 = 12704355194.584\dots \quad (4)$$

The cubic $x^3 + Ax + B = 0$ has one real root $e_1 \approx -3704.0$ and a complex conjugate pair with imaginary part $\pm 8.68 \times 10^{-5}$. The roots are separated by only 1.74×10^{-4} — the curve is nearly degenerate.

The period integral, after the substitution $x = e_1 + t^2$, becomes

$$\Omega = \int_0^{\infty} 2 \, dt / \sqrt{(t^4 + ct^2 + d)}$$

where $c = 3e_1$ and $d = 3e_1^2 + A$. The polynomial $t^4 + ct^2 + d$ has a minimum at $t = \sqrt{-c/2} \approx 74.54$, where its value is

$$d - c^2/4 = 7.53 \times 10^{-9}$$

The integrand therefore has a spike of height $\approx 23,000$ and width $\approx 10^{-5}$ at $t \approx 74.54$. Standard quadrature — whether Simpson’s rule, Gaussian, or even adaptive Gauss-Kronrod — walks right past this spike, producing answers that are wrong by factors of 2-5.

The resolution is multi-point splitting: divide the integration domain at points clustered around the spike location, with spacing that narrows from 1 to 10^{-5} approaching the spike, and apply high-order quadrature (maxdegree 15) to each sub-interval independently. With 80-digit intermediate arithmetic via mpmath, this achieves

$$\Omega = 0.53808589097967547733393545140068... \quad (7)$$

in agreement with the LMFDB value to 30 significant figures.

§4 Results

§4.1 BSD assembly

Combining all derived quantities:

$$|Sha| = L(E, 1) / (\Omega \cdot \prod c_p) = 6.457030691756 / (0.538085890980 \times 3) = 4.0000000000$$

This is $4 = 2^2$: a positive perfect square, as implied by the Cassels-Tate pairing [5], which forces $|Sha|$, if finite, to be a perfect square.

§4.2 The extended table

We now extend Table 1 of Paper 143 with the $|Sha| = 4$ column. The fake columns are generated by perturbing a_p and then rebuilding the full multiplicative sequence a_n from the perturbed primes (not merely poking at prime indices). This ensures the fake L-series is fully consistent with the perturbed controlling function.

Genuine 389a1	Genuine 27606c1	Random noise	Systematic +1

Sha	1.000000	4.000000	3.429	5.023
Perfect square?	Yes ($1 = 1^2$)	Yes ($4 = 2^2$)	No	No
Group order ≥ 1 ?	Yes	Yes	Yes	Yes
All ingredients derived?	Yes	Yes	N/A	N/A

Table 1. The extended phantom filter comparison. Both genuine curves produce perfect squares. Both fake inputs produce non-integers. The fake sequences are fully rebuilt from perturbed a_p via Hecke multiplicativity, so every composite a_n is consistent with the perturbed controlling function. The functional equation holds for all four inputs (verified in Paper 143 [2]): the FE is architectural, not selective.

§4.3 The derivation chain

For the record, the status of every quantity in the computation:

Quantity	Method	Status	Agreement with LMFDB	
a_p	Point counting mod p	DERIVED	8/8 exact	
$L(E, 1) = 6.457031$	Convergent series (5000 terms)	DERIVED	16 figures	significant
$\Pi c_p = 3$	Tate algorithm (semistable)	DERIVED	Exact	
$\Omega = 0.538086$	80-digit quadrature, spike-split	DERIVED	30 figures	significant
$ \text{Sha} = 4$	BSD quotient	DERIVED	Exact (to 10 digits)	

No quantity uses an external value as input. All cross-checks against the LMFDB are performed *after* the computation, not during it.

§5 Discussion

§5.1 *What genuine phantoms are*

An element of $\text{Sha}(E/\mathbb{Q})$ is a torsor for E that has a rational point over every local field but no global rational point. From the compiler’s perspective, a Sha element affects the right-hand side of the BSD formula: the ratio $L(E, 1)/(\Omega \cdot R \cdot \prod c_p/|\text{tor}|^2)$ is not 1 but $|\text{Sha}| = 4$. The “excess” in the L-value — the factor by which the analytic side exceeds what the geometric side accounts for — is a perfect square.

This is the distinction that Table 1 makes visible. A fake perturbation to the controlling function — with the full multiplicative sequence rebuilt from perturbed a_p — produces an excess that is not a perfect square (3.429, 5.023). A genuine Sha element produces an excess that is a perfect square ($4 = 2^2$). The compiler can tell the difference because genuine phantoms arise from the arithmetic of the curve — they are torsors, algebraic objects with Galois structure — while fake perturbations arise from nothing.

§5.2 *The role of the near-degenerate period*

The computational difficulty of the period integral is itself informative. The near-degeneracy of 27606c1 (roots separated by 10^{-4}) means the elliptic curve is close to a singular curve — close to the boundary of the moduli space. The spike in the integrand is the period integral’s way of registering this proximity: as the roots coalesce, the period diverges logarithmically. Standard numerical methods fail precisely because they do not resolve the geometry near the boundary.

That the compiler still produces $|\text{Sha}| = 4.0000000000$ to ten digits — once the spike is properly resolved — is evidence that the integer-output mechanism is robust. It works not only for well-separated curves like 389a1 but also for near-degenerate curves where the numerics are challenging. The integrality is a property of the mathematics, not of the numerical convenience of the inputs.

§5.3 Strengthening the phantom filter

Paper 143 argued that the BSD compiler ignores fake input: the functional equation holds, every step completes, but the output is non-integer. The present paper adds the converse: the compiler *recognises* genuine input. When Sha elements are present, the compiler produces their count (as a perfect square). This is not a test the compiler runs. It is a property of the output space: topology in, integer out. More topology in ($\text{Sha} \neq 1$), larger integer out. No topology in, non-integer out.

The three-way comparison is now complete:

Genuine, Sha = 1: the compiler outputs 1. (Paper 142)

Genuine, Sha = 4: the compiler outputs 4. (This paper)

Fake: the compiler outputs a non-integer. (Paper 143)

§6 What Remains Open

(a) Extension to $|\text{Sha}| = 9$, the smallest odd perfect square appearing in Cremona's tables, which would test the compiler on 3-torsion Sha elements rather than the 2-torsion seen here — a genuinely different arithmetic mechanism.

(b) Tate's algorithm for additive reduction (primes $p = 2, 3$ with ~ 15 sub-cases), which would open the compiler to non-semistable curves and vastly more examples.

(c) A rigorous proof that genuine Sha elements always produce perfect-square output — i.e., a proof that $|\text{Sha}|$ is always a perfect square, Cassels's conjecture.

References

- [1] Keeble, C. (2026). The Finite Gap: BSD at Rank 2 Reduces to Finiteness of Sha. *Bootstrap Universe Programme*, Paper 142. DOI: 10.5281/zenodo.19090811.

- [2] Keeble, C. (2026). The Phantom Filter: Why Fake Controlling Functions Produce Non-Integer Output. *Bootstrap Universe Programme*, Paper 143. DOI: 10.5281/zenodo.19096472.
- [3] Cremona, J. E. (1997). *Algorithms for Modular Elliptic Curves*, 2nd edition. Cambridge University Press.
- [4] LMFDB Collaboration (2024). The L-functions and Modular Forms Database. <https://www.lmfdb.org/> — Curve 27606.c1.
- [5] Cassels, J. W. S. (1962). Arithmetic on curves of genus 1, IV. *J. Reine Angew. Math.* 211, 95–112.
- [6] Silverman, J. H. (2009). *The Arithmetic of Elliptic Curves*, 2nd edition. Springer.
- [7] Keeble, C. (2026). The Boot Sequence: BSD at Rank 0 on the Compiler’s First Curve. *Bootstrap Universe Programme*, Paper 141. DOI: 10.5281/zenodo.19084623.

Appendix A Test Code

The following self-contained Python script computes every quantity in this paper from the Weierstrass coefficients. It requires `mpmath` for the high-precision period computation. Running time: approximately 2 seconds.

```
#!/usr/bin/env python3
"""Paper 144 Test: |Sha| = 4 Through the BSD Compiler
Curve: 27606c1. Dependencies: mpmath"""
import math, random, time
from fractions import Fraction

A1, A2, A3, A4, A6 = 1, 0, 0, -10289707, 12703497719
CONDUCTOR = 27606
BAD_PRIMES = [2, 3, 43, 107]

def compute_ap(a1, a2, a3, a4, a6, p):
    if p == 2:
        count = 1
        for x in range(2):
            for y in range(2):
                if (y*y+a1*x*y+a3*y)%2 == (x**3+a2*x*x+a4*x+a6)%2:
                    count += 1
        return p + 1 - count
    total = 0
    for x in range(p):
        b = (a1*x+a3)%p; c = (x**3+a2*x*x+a4*x+a6)%p
        D = (b*b+4*c)%p
        if D != 0:
            ls = pow(D, (p-1)//2, p)
            total += (-1 if ls == p-1 else 1)
    return -total

def sieve_primes(limit):
    s = [True]*(limit+1); s[0]=s[1]=False
    for i in range(2, int(limit**0.5)+1):
        if s[i]:
            for j in range(i*i, limit+1, i): s[j]=False
```

```

    return [p for p in range(2, limit+1) if s[p]]

def rebuild_an_from_ap(ap_dict, bad, limit):
    """Build full a_n from a_p dict via Hecke multiplicativity."""
    primes = sieve_primes(limit); bad_set = set(bad)
    an = [0]*(limit+1); an[1] = 1; apk = {}
    for p in primes:
        apk[(p,1)] = ap_dict[p]; pk, k = p, 1
        while pk*p <= limit:
            pk *= p; k += 1
            if p in bad_set: apk[(p,k)] = ap_dict[p]**k
            elif k == 2: apk[(p,k)] = ap_dict[p]**2 - p
            else: apk[(p,k)] = ap_dict[p]*apk[(p,k-1)] - p*apk[(p,k-2)]
    for n in range(2, limit+1):
        val, temp = 1, n
        for p in primes:
            if p*p > temp: break
            if temp%p == 0:
                k = 0
                while temp%p == 0: temp //= p; k += 1
                val *= apk[(p,k)]
            if temp > 1: val *= ap_dict[temp]
        an[n] = val
    return an

def compute_L(an, N):
    sqrtN = math.sqrt(N); decay = 2*math.pi/sqrtN
    return 2.0*sum(an[n]/n*math.exp(-decay*n)
                   for n in range(1,len(an)) if an[n]!=0)

def compute_tamagawa(a1,a2,a3,a4,a6, bad, ap):
    b2=a1**2+4*a2; b4=a1*a3+2*a4; b6=a3**2+4*a6
    b8=a1*a1*a6+4*a2*a6-a1*a3*a4+a2*a3*a3-a4*a4
    D = -b2**2*b8 - 8*b4**3 - 27*b6**2 + 9*b2*b4*b6
    prod = 1
    for p in bad:
        n, t = 0, abs(D)
        while t%p==0: t//=p; n+=1
        cp = n if ap[p]==1 else (1 if n%2==1 else 2)
        prod *= cp
    return prod

def compute_period(a1,a2,a3,a4,a6):
    from mpmath import mp, mpf, sqrt, quad, inf, sign, cbrt
    mp.dps = 80
    b2=a1**2+4*a2; b4=a1*a3+2*a4; b6=a3**2+4*a6
    c4=b2**2-24*b4; c6=-b2**3+36*b2*b4-216*b6
    A = mpf(-c4)/48; B = mpf(-c6)/864
    x = -sign(B)*cbrt(abs(B)) # general initial guess
    for _ in range(200):
        f=x**3+A*x+B; fp=3*x**2+A; x-=f/fp
    e1=x; c_ =3*e1; d_ =3*e1**2+A
    M=-c_/2; R=d_-c_**2/4; tm=sqrt(M)
    def integ(t): return 2/sqrt((t**2-M)**2+R)
    pts=[mpf(0)]
    for o in [1, 0.1, 0.001, 0.00001]:
        pts.append(tm-o)
    pts.append(tm)
    for o in [0.00001, 0.001, 0.1, 1]:
        pts.append(tm+o)
    pts += [5*tm, inf]
    tot = sum(quad(integ,[pts[i],pts[i+1]],maxdegree=15)
              for i in range(len(pts)-1))
    return float(tot)

# Run
LIMIT = 5000

```

```

primes = sieve_primes(LIMIT)
ap = {p: compute_ap(A1,A2,A3,A4,A6,p) for p in primes}
an = rebuild_an_from_ap(ap, BAD_PRIMES, LIMIT)
L = compute_L(an, CONDUCTOR)
tam = compute_tamagawa(A1,A2,A3,A4,A6,BAD_PRIMES,ap)
omega = compute_period(A1,A2,A3,A4,A6)
sha = L / (omega * tam)
print(f"|Sha| = {sha:.10f}") # 4.0000000000
print(f" = {round(sha)} = {round(sha**0.5)}^2") # = 4 = 2^2

```

Bootstrap Universe Programme · Paper 144 · v1.0

© 2026 Dr. Clifford Keeble · Woodbridge, UK

